33rd JK Conference Korean Workshop
Session 2

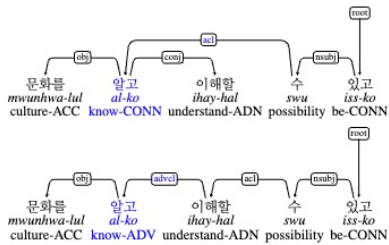
Hakyung Sung
University of Hawaii at Manoa

LCLB G20, UIUC

August 6, 2026

Session 1 (Review)

- Overview of UD's goals and design
- Elements of UD annotations
- Korean UD corpora
- Annotation decisions and criteria
  - See our recent paper *Parser agreement/disagreement in L2 Korean UD*



Topics to cover

- Intro (5 mins)
- Annotate Korean texts (20 mins)
- Calculate frequencies (20 mins)
- Generate dependency-based concordance lines (20 mins)
- Conclusion (5 mins)
- NOT to understand every details of Python

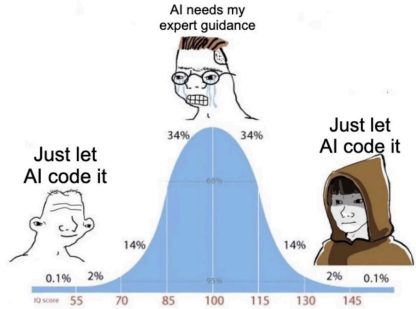
GitHub Repository:
https://github.com/hksung/33JKConference_KoreanWorkshop

Colab Notebook: https://colab.research.google.com/drive/1RJdvKyT_bJPM6M0EFZ2acxUe-feehdCn?usp=sharing

* *Please save blue a copy of Notebook to your Drive before starting the session.*

Colab notebook

- You do not need to write the Python code from scratch; the notebook already provides it (Simply click the ▶ button to run each code cell)
- To apply the workflow to your own research, you might need to modify parts of the code
- AI tools can be helpful for adapting/generating code; BUT effective prompting still requires a minimal understanding of what the code is doing



1. Annotating Korean Texts

with *Stanza*

What we will do

- Install and import the required Python packages
- Download the Korean GSD model
- Build a Korean NLP pipeline
- Load two sample Korean texts
- Generate UD annotations

1.1. Install and import the required packages

- Run the following cell in Colab:

```
!pip install stanza pandas
```

- `stanza` provides tools for tokenization, part-of-speech tagging, lemmatization, and dependency parsing
  - `https://stanfordnlp.github.io/stanza/`
- `pandas` allows us to organize annotations as tables/dataframes

What does `pip install` mean?
`pip install` downloads and installs Python packages that are not already available in the Colab *environment*.

What does `!` mean?
The exclamation mark tells Colab to run the line as a terminal command rather than as regular Python code.

1.1. Install and import the required packages

```
import re
from pathlib import Path
from IPython.display import display

import stanza
import pandas as pd
```

Why do we import packages?

Importing makes tools from built-in and newly installed Python libraries available in the notebook.

- `re`: finds/splits text patterns
- `Path`: works with file/folder paths
- `display`: displays tables neatly in Colab
- `stanza`, `pandas`

1.2. Download the Korean GSD model and build the pipeline

```
PROCESSORS = "tokenize,pos,lemma,depparse"

stanza.download(
    lang="ko",
    package="gsd",
    processors=PROCESSORS,
    verbose=False
)

nlp = stanza.Pipeline(
    lang="ko",
    package="gsd",
    processors=PROCESSORS,
    tokenize_no_ssplit=True,
    use_gpu=False,
    verbose=False
)
```

What does this code do?

- Downloads Stanza's Korean GSD model
- Builds a pipeline, stored as `nlp`
- Applies four annotation steps:
  1. `tokenize`
  2. `pos`
  3. `lemma`
  4. `depparse`

1.3. Load and inspect two sample texts

```
import requests

BASE_URL = (
    "https://raw.githubusercontent.com/"
    "hksung/33JKConference_KoreanWorkshop/"
    "main/data"
)
```

What does this code do?

- `requests`: downloads files from a web address
- `BASE_URL`: stores the shared location of the sample files
- The files are downloaded directly from the workshop GitHub repository

1.3.2. Load and inspect two sample texts

```
def load_raw_sentences(filename):
    url = f"{BASE_URL}/{filename}"

    response = requests.get(url)
    response.raise_for_status()

    text = response.text.replace("\n", " ")

    sentences = [
        sentence.strip()
        for sentence in re.split(
            r"(?<=[.!?])\s+", text
        )
        if sentence.strip()
    ]

    return sentences
```

What does this function do?

- Receives a filename
- Downloads the corresponding text file
- Replaces line breaks with spaces
- Splits the text after sentence-final punctuation
- Returns a list of sentences

1.3.3. Load and inspect two sample texts

```
raw_sentences1 = load_raw_sentences("sample1.txt")
raw_sentences2 = load_raw_sentences("sample2.txt")

for sample_name, sentences in [
    ("Sample 1", raw_sentences1),
    ("Sample 2", raw_sentences2)
]:
    print(
        f"{sample_name} | "
        f"Number of sentences: {len(sentences)}"
    )

    for i, sentence in enumerate(sentences, start=1):
        print(f"{i}. {sentence}")

    print()
```

1.4.1. Apply UD-based annotation

```
def annotate_sentences(
    raw_sentences,
    document_label
):
    rows = []

    for sentence_id, sentence_text \
        in enumerate(raw_sentences, start=1):

        doc = nlp(sentence_text)
```

What does this function do?

- Input: a list of sentences
- Assigns an ID to each sentence
- Applies the NLP pipeline to each sentence
- Creates `rows` to store the results

1.4.2. Apply UD-based annotation

```
doc = nlp(sentence_text)
for sent in doc.sentences:
    words_by_id = {
        word.id: word
        for word in sent.words
    }
    for word in sent.words:
        rows.append(
            { ...
                "text": word.text,
                "lemma": word.lemma,
                "upos": word.upos,
                "xpos": word.xpos,
                ...
            }
        )
```

Stanza output

`Document` → `Sentence` → `Word`

Word attributes (example)

- `text`: surface form, typically an eojeol (학생이)
- `lemma`: morphemes combined with + (학생+이)
- `upos`: Universal POS tag (`NOUN`)
- `xpos`: Korean-specific morpheme-level POS tags (`NNG+JKS`) ▸ Sejong tag set)

1.4.3. Apply UD-based annotation

```
annotations1 = annotate_sentences(
    raw_sentences1,
    "text_1"
)

annotations2 = annotate_sentences(
    raw_sentences2,
    "text_2"
)
```

What does this code do?

- Applies the same annotation function to both texts
- Stores the results in two separate tables
- Adds a document label to every annotated word

1.4.4. Select columns for the preview

```
preview_cols = [
    "sentence",
    "sentence_id",
    "word_id",
    "text",
    "lemma",
    "upos",
    "xpos",
]
```

What does this code do?

- Selects the columns to show in the annotation preview
- Stores the selected column names in `preview_cols`

1.4.5. Preview the annotations

```
print("Annotation for Text 1")
display(
    annotations1[preview_cols].head(20)
)

print("Annotation for Text 2")
display(
    annotations2[preview_cols].head(20)
)
```

What does this code do?

- Selects only the columns listed in `preview_cols`
- Displays the first 20 annotated words
- Allows us to inspect the annotations before continuing

Activity 1: Inspect the annotations

- Change the following values to inspect another text or sentence:
  - `TEXT_TO_INSPECT = 1`: select Text 1
  - `SENTENCE_TO_INSPECT = 2`: select Sentence 2

- Compare the `upos` and `xpos` tags ▸ Sejong tag set

- Discuss any interesting, unexpected, or unclear annotations with the person next to you. **Keep in mind that these sentences are drawn from L2 Korean learner writing data** ▸ L2KARG corpus

1.5. Save the output into conllu files

```
...
def save_conllu(raw_sentences, text_label, output_path):
    output_path = Path(output_path)
    conllu_lines = []
    for sentence_id, sentence_text in enumerate(raw_sentences, start=1):
        doc = nlp(sentence_text)
        for stanza_sentence_id, sent in enumerate(doc.sentences, start=1):
            conllu_lines.append(
                f"# sent_id = {text_label}-{sentence_id}-{stanza_sentence_id}")
            conllu_lines.append(
                f"# text = {sent.text or sentence_text}")
            for word in sent.words:
                fields = [
                    word.id, word.text, word.lemma, word.upos, word.xpos,
                    word.feats, word.head, word.deprel, word.deps, word.misc]
                conllu_lines.append(
                    "\t".join(conllu_value(field) for field in fields))
            conllu_lines.append("")
            ...
```

1.5. Save the output into conllu files

```
save_conllu(
    raw_sentences=raw_sentences1,
    text_label="text1",
    output_path="annotations1.conllu")

save_conllu(
    raw_sentences=raw_sentences2,
    text_label="text2",
    output_path="annotations2.conllu")
```

Why save the annotations?

- Preserve the annotations after running the code
- Share/analyze them using other tools
- How can we download the files to a local computer from Colab?

2. Calculate frequencies

raw and normalized feature frequencies

What we will do

- Combine the two annotation DataFrames
- Exclude punctuation and define normalization denominators
- Create a reusable frequency-calculation function
- Calculate raw and normalized UPOS frequencies
- Compare UPOS patterns across the two learner texts
- Calculate and visualize Korean-specific XPOS frequencies

2.1. Combine the annotation DataFrames

```
annotations = pd.concat(
    [annotations1, annotations2],
    ignore_index=True)

...

display(annotations.head(20))
```

What does this code do?

- Combines `annotations1` and `annotations2` into one DataFrame
- Stores the combined table as `annotations`
- `ignore_index=True` creates a new continuous row index

2.2. Define the normalization denominator

```
analysis_words = annotations.loc[
    annotations["upos"] != "PUNCT"
].copy()

word_counts_by_text = (
    analysis_words
    .groupby("document")
    .size()
    .reset_index(name="total_words")
)

if (word_counts_by_text["total_words"] == 0).any():
    raise ValueError(
        "At least one text has no "
        "non-punctuation syntactic words."
    )

print("Normalization denominators:")
display(word_counts_by_text)
```

What does this code do?

- Excludes punctuation from the frequency analysis
- Counts the number of remaining words in each text
- Uses these word counts as normalization denominators

Why normalize frequencies?

2.2. Define the normalization denominator

```
analysis_words = annotations.loc[
    annotations["upos"] != "PUNCT"
].copy()

word_counts_by_text = (
    analysis_words
    .groupby("document")
    .size()
    .reset_index(name="total_words")
)

if (word_counts_by_text["total_words"] == 0).any():
    raise ValueError(
        "At least one text has no "
        "non-punctuation syntactic words."
    )

print("Normalization denominators:")
display(word_counts_by_text)
```

What does this code do?

- Excludes punctuation from the frequency analysis
- Counts the number of remaining words in each text
- Uses these word counts as normalization denominators

Why normalize frequencies?

Raw frequencies are affected by text length. Normalization allows us to compare features across texts of different lengths.

2.3. Create a reusable frequency function

```
def calculate_frequency(
    data, feature_column, denominator):

    if denominator == 0:
        raise ValueError(
            "The denominator is 0. ")

    frequency = (
        data[feature_column]
        .fillna("_")
        .value_counts()
        .rename_axis(feature_column)
        .reset_index(name="raw_frequency"))

    frequency["per_1,000_words"] = (
        frequency["raw_frequency"]
        / denominator
        * 1000
    )
    return frequency
```

What does this function do?

- Counts each value in a selected annotation column
- Stores the counts as `raw_frequency`
- Calculates normalized frequency per 1,000 words
- Returns the results as a new DataFrame

Normalization formula

$$\frac{\text{raw frequency}}{\text{total words}} \times 1{,}000$$

Activity 2. Calculate UPOS frequencies

```
upos_frequency_list = []

for document, data_subset \
    in analysis_words.groupby("document"):

    denominator = (
        TOTAL_WORDS_BY_TEXT[document]
    )

    frequency = calculate_frequency(
        data=data_subset,
        feature_column="upos",
        denominator=denominator,
    )

    frequency["document"] = document
    upos_frequency_list.append(frequency)
```

What does this code do?

- Separates the annotations by document
- Retrieves each text's normalization denominator
- Calculates raw and normalized UPOS frequencies
- Adds the document label to each frequency table
- Stores the results in `upos_frequency_list`

```
upos_frequency = pd.concat(
    upos_frequency_list,
    ignore_index=True)

upos_frequency = upos_frequency[
    [
        "document",
        "upos",
        "raw_frequency",
        "per_1,000_words",
    ]]

upos_frequency = (
    upos_frequency
    .sort_values(
        by=["document", "upos"]
    )
    .reset_index(drop=True))

display(upos_frequency)
```

What does this code do?

- Combines the frequency results from both texts and displays a table

Output columns

- `document`: Text 1 or 2?
- `upos`: UPOS tag
- `raw_frequency`: observed count
- `per_1,000_words`: normalized frequency

Discussion questions

Text information

Text 1 was written by a lower-proficiency learner, whereas Text 2 was written by a higher-proficiency learner.

1. What preliminary hypothesis can you draw about the two learners' use of UPOS categories?
2. If you only consider `raw_frequency`, which text appears to use more nouns, verbs, adverbs, and pronouns?
3. Does the interpretation change when you compare `normalized_frequency`?
4. Why might raw frequency be misleading when comparing texts of different lengths?

Activity 3. Calculate XPOS frequencies

- XPOS provides detailed Korean-specific part-of-speech information
- One syntactic word may contain multiple XPOS tags
  - `VV+ETM` $\rightarrow$ `VV` and `ETM`
- We should first split these tag sequences before calculating frequencies.

```
xpos_rows = []

for _, row in analysis_words.iterrows():
    xpos_tags = str(
        row["xpos"]
    ).split("+") # SPLIT!

    for xpos_tag in xpos_tags:
        xpos_rows.append(
            {
                "document": row["document"],
                "xpos": xpos_tag,
            }
        )

xpos_long = pd.DataFrame(xpos_rows)
```

Output

`xpos_long` contains one row for each individual XPOS tag.

```
TOTAL_XPOS_BY_TEXT = (
    xpos_long
    .groupby("document")
    .size()
    .to_dict())

xpos_frequency_list = []

for document, data_subset \
    in xpos_long.groupby("document"):

    denominator = (TOTAL_XPOS_BY_TEXT[document])

    frequency = calculate_frequency(
        data=data_subset,
        feature_column="xpos",
        denominator=denominator,)

    frequency["document"] = document
    xpos_frequency_list.append(frequency)
```

What does this code do?

- The denominator is the total number of individual XPOS tags (i.e., XPOS frequency is normalized per 1,000 morphemes)
- The same reusable frequency function from Activity 1 is applied

```
xpos_frequency = pd.concat(
    xpos_frequency_list,
    ignore_index=True)

...

xpos_frequency = xpos_frequency.rename(
    columns={
        "per_1,000_words":
        "per_1,000_xpos_tags"})

xpos_frequency = (
    xpos_frequency
    .sort_values(
        by=["document", "xpos"])
    .reset_index(drop=True))

display(xpos_frequency)
```

Repeated from Activity 1

Combining, reordering, sorting, and displaying the results follow the same workflow as the UPOS analysis.

```
import matplotlib.pyplot as plt

TOP_N = 15

xpos_comparison = (
    xpos_frequency
    .pivot(
        index="xpos",
        columns="document",
        values="per_1,000_xpos_tags")
    .fillna(0))

xpos_comparison["total"] = (xpos_comparison.sum(axis=1))

top_xpos_comparison = (
    xpos_comparison
    .sort_values(
        "total",
        ascending=False)
    .head(TOP_N)
    .drop(columns="total"))
```

What does this code do?

- Reshapes the normalizedfrequency table for plotting
- Selects the *15* most frequent XPOS tags across both texts

You may modify

Change `TOP_N` to display more or fewer XPOS tags.

```
top_xpos_comparison = (
    top_xpos_comparison.rename(
        columns={
            "text_1": "Text 1 (lower proficiency)",
            "text_2": "Text 2 (higher proficiency)",}))

...

top_xpos_comparison = (
    top_xpos_comparison.loc[plot_order])

ax = top_xpos_comparison.plot(
    kind="barh",
    figsize=(10, 8))

ax.set_xlabel("Frequency per 1,000 XPOS tags")
ax.set_ylabel("XPOS tag")
ax.legend(title="Text")
plt.tight_layout()
plt.show()
```

What does this plot do?

- Renames the two text labels
- Orders the tags by combined frequency
- Creates a horizontal bar plot
- Compares normalized XPOS frequencies across the two texts

Discussion questions

1. Which Korean-specific XPOS tags occur most frequently in general?
2. Which tags seem to distinguish the two proficiency levels, and what linguistic factors might explain the differences?
3. Beyond characterizing proficiency, how could XPOS annotations support research questions in your own area of interest?

Extra activity: Try new texts

- If you have extra time, return to previous sections and explore `sample3.txt` and `sample4.txt`
- Repeat the same workflow:
  - annotate the texts
  - inspect the POS tags
  - calculate normalized frequencies
  - visualize the XPOS patterns
- Samples 3 and 4 respond to a different prompt: *Should children start learning a foreign language at an early age?*

3. Generate dependency-based concordance lines

explore linguistic features in context

Concordance lines

- Frequency analysis tells us how often a linguistic feature occurs.
- However, frequency alone does not show how the feature is used **in context**.
- Concordance lines allow us to inspect recurring linguistic patterns in their original sentences.

What we will do

- Add head-word information to the dependency annotations
- Select a dependency relation ▸ UD tags
- Generate dependent–head concordance lines for the selected relation
- Select a dependent keyword and inspect the predicates it attaches to
- Compare predicate contexts and morphosyntactic patterns across texts

3.1. Load and annotate two new texts

```
raw_sentences5 = load_raw_sentences(
    "sample5.txt"
)

raw_sentences6 = load_raw_sentences(
    "sample6.txt"
)

annotations5 = annotate_sentences(
    raw_sentences5,
    "text_5"
)

annotations6 = annotate_sentences(
    raw_sentences6,
    "text_6"
)
```

What does this code do?

- Loads Samples 5 and 6 from the workshop repository
- Reuses the functions defined in Section 1
- Applies the Korean annotation pipeline to both texts
- Stores the results as `annotations5` and `annotations6`

3.1. Load and annotate two new texts (Check)

```
print(
    f"Text 5: "
    f"{len(raw_sentences5)} "
    f"input sentences"
)

print(
    f"Annotated syntactic words: "
    f"{len(annotations5):,}"
)

print(
    f"\nText 6: "
    f"{len(raw_sentences6)} "
    f"input sentences"
)

print(
    f"Annotated syntactic words: "
    f"{len(annotations6):,}"
)
```

3.2. Combine the annotation tables

```
annotations56 = pd.concat(
    [annotations5, annotations6],
    ignore_index=True
)

print(
    f"\nTotal annotated syntactic words: "
    f"{len(annotations56):,}"
)

display(annotations56.head(20))
```

3.3. Add the XPOS tag of each word's syntactic head

```
def add_head_xpos(data):
    output_rows = []
    for _, sentence_df in data.groupby(
        ["document", "sentence_id"],
        dropna=False, sort=False,):
        words_by_id = (sentence_df.set_index("word_id")
            .to_dict("index"))
        for _, row in sentence_df.iterrows():
            head_id = row["head"]
            if head_id == 0:
                head_xpos = "ROOT"
            else:
                head_word = words_by_id.get(head_id)
                if head_word is None:
                    head_xpos = "_"
                else:
                    head_xpos = (head_word["xpos"])
            row_dict = row.to_dict()
            row_dict["head_xpos"] = head_xpos
            output_rows.append(row_dict)
    return pd.DataFrame(output_rows)
```

What does this function do?

- Retrieves the head word's Korean-specific POS tag
- Adds this information as a new `head_xpos` column

3.3. Add the XPOS tag of each word's syntactic head

```
annotations56_with_heads = (
    add_head_xpos(annotations56)
)

head_preview_columns = [
...
    "head_upos",
    "head_xpos",
]

display(
    annotations56_with_heads[
        head_preview_columns
    ].head(20)
)
```

3.4. Select a dependency relation

```
TARGET_RELATION = "nsubj"
```

- `nsubj`: nominal subject
- `obj`: object
- `obl`: oblique nominal
- `advcl`: adverbial clause
- `advmod`: adverbial modifier
- `nmod`: nominal modifier

What does this code do?

Selects the dependency relation that we want to investigate.

Example

With `nsubj`, we can examine which subjects attach to which heads (i.e., predicates) (we will do this now!)

3.4.1. Generate concordance lines

```
concordance = annotations56_with_heads.loc[
    annotations56_with_heads["deprel"]
    == TARGET_RELATION,
    [
        "sentence",
        "document",
        "sentence_id",
        "word_id",
        "text",
        "lemma",
        "upos",
        "xpos",
        "head",
        "deprel",
        "head_text",
        "head_lemma",
        "head_upos",
        "head_xpos",
    ],
].copy()
```

What does this code do?

- Keeps only rows with the selected relation
- Retains information about the dependent, head, and sentence

3.4.1. Generate concordance lines

```
concordance = concordance.rename(
    columns={
        "text": "dependent",
        "lemma": "dependent_lemma",
        "upos": "dependent_upos",
        "xpos": "dependent_xpos",
        "deprel": "relation",})

concordance = (
    concordance
    .sort_values(
        [
            "document",
            "dependent_lemma",
            "head_lemma",
            "sentence_id",])
    .reset_index(drop=True)
)

print(f"Target relation: {TARGET_RELATION}")
print(f"Number of concordance lines: "
      f"{len(concordance):,}")
```

What does this code do?
Renames the dependent columns, sorts similar patterns together, and displays the concordance *table*.

One row represents

$$\texttt{dependent} \xrightarrow{\texttt{relation}} \texttt{head}$$

3.4.2. Identify frequent dependents

```
dependent_patterns = (
    concordance
    .groupby(
        [
            "document",
            "dependent_lemma",
            "dependent_upos",
            "dependent_xpos",
        ],dropna=False,)
    .size()
    .reset_index(name="frequency")
    .sort_values(
        [
            "document",
            "frequency",
            "dependent_lemma",
        ],
        ascending=[True, False, True],)
    .reset_index(drop=True))

display(dependent_patterns.head(30))
```

What does this code do?
Counts which dependent lemmas occur most frequently with the selected relation in each text.

3.4.3. Compare dependent lemmas

```
dependent_pattern_comparison = (
    dependent_patterns
    .pivot_table(
        index=[
            "dependent_lemma",
            "dependent_upos",
            "dependent_xpos",
        ],
        columns="document",
        values="frequency",
        fill_value=0,
        aggfunc="sum",
    )
    .reset_index()
)

dependent_pattern_comparison[
    "total"
] = (
    dependent_pattern_comparison["text_5"]
    + dependent_pattern_comparison["text_6"]
)
```

What does this code do?
Places the dependent frequencies from Texts 5 and 6 side by side for comparison.

3.4.3. Compare dependent lemmas

```
dependent_pattern_comparison = (
    dependent_pattern_comparison
    .sort_values(
        "total",
        ascending=False
    )
    .reset_index(drop=True)
)

display(
    dependent_pattern_comparison
    .drop(columns="total")
    .head(20)
)
```

What does this code do?
Sorts dependent lemmas by their combined frequency and displays the top 20.

3.4.4. Select a dependent keyword

```
TARGET_DEPENDENT_KEYWORD = (
    dependent_pattern_comparison
    .iloc[0]["dependent_lemma"]
    .split("+")[0]
)

TARGET_DEPENDENT_KEYWORD = "학생"
```

What does this code do?

Selects one dependent keyword for closer inspection.

Why use a keyword?

학생 can match lemmas such as 학생+이, 학생+은, and 학생+들+이, which all store in one eojeol.

3.4.5. Add compact predicate context

- The syntactic head alone may not capture the complete Korean predicate
- The function first locates the head using its numeric `head` ID
- It then checks up extra information:
  - auxiliary predicates (e.g., 있다, 되다)
  - bridge nouns (e.g., 수, 것)
  - negative expressions (e.g., 않다, 못하다, 없다)
- Relevant tokens are added to create a compact predicate context

3.4.5. What does the function create?

Predicate information

- `predicate_context`: surface forms
- `predicate_context_lemmas`: lemma sequence
- `predicate_context_xpos`: XPOS sequence
- `polarity`: affirmative-like, negative-like, root, or unknown

Morphosyntactic pattern

The function combines the dependent XPOS, dependency relation, and predicate XPOS:

$$\texttt{dependent XPOS} \xrightarrow{\texttt{relation}} \texttt{predicate XPOS}$$

e.g.,

$$\texttt{NNG+JKS} \xrightarrow{\texttt{nsubj}} \texttt{VV+EC + VX+EF}$$

3.4.5. Example predicate contexts

- 비교하다 — head predicate only
- 비교하지 않다 — predicate + negation
- 경험할 수 있다 — predicate + bridge noun + auxiliary
- 배우게 되다 — predicate + auxiliary continuation
- The function uses a simple rule-based heuristic!

3.4.6. Inspect the selected dependent keyword

```
selected_dependent_examples = (
    concordance.loc[
        concordance["dependent_lemma"]
        .fillna("")
        .str.contains(
            TARGET_DEPENDENT_KEYWORD,
            regex=False,
        )
    ]
    .sort_values(
        [
            "document",
            "dependent_lemma",
            "head_lemma",
            "sentence_id",
        ]
    )
    .reset_index(drop=True)
)
```

What does this code do?
Keeps only concordance lines whose dependent lemma contains the selected keyword.

3.4.6. Inspect the selected dependent keyword

```
selected_dependent_examples = (
    add_predicate_context(
        concordance_data=
            selected_dependent_examples,
        full_data=
            annotations56_with_heads,))

print(f"Target relation: "
      f"{TARGET_RELATION}")

print(f"Selected dependent keyword: "
      f"{TARGET_DEPENDENT_KEYWORD}")

print(f"Number of matching lines: "
      f"{len(selected_dependent_examples):,}")

display(
    selected_dependent_examples[
        selected_example_columns
    ]
)
```

What does this code do?
Adds predicate-context information and displays all matching concordance lines.

3.4.7. Compare predicate contexts

```
predicate_context_patterns = (
    selected_dependent_examples
    .groupby(
        [
            "document",
            "predicate_context_lemmas",
            "predicate_context",
            "predicate_context_xpos",
            "polarity",
        ],
        dropna=False,
    )
    .size()
    .reset_index(name="frequency")
)
```

What does this code do?
Counts which predicate contexts occur with the selected dependent in each text.

3.4.7. Compare predicate contexts

```
predicate_context_comparison = (
    predicate_context_patterns
    .pivot_table(
        index=[
            "predicate_context_lemmas",
            "predicate_context",
            "predicate_context_xpos",
            "polarity",],
        columns="document",
        values="frequency",
        fill_value=0,
        aggfunc="sum",)
    .reset_index()
)

predicate_context_comparison[
    "total_frequency"
] = (predicate_context_comparison[
        document_columns
    ].sum(axis=1)
)
```

What does this code do?
Compares predicate contexts across Texts 5 and 6 and calculates their combined frequency.

3.4.8. Compare morphosyntactic patterns

```
morphosyntactic_patterns = (
    selected_dependent_examples
    .groupby(
        [
            "document",
            "morphosyntactic_pattern",
        ],
        dropna=False,
    )
    .size()
    .reset_index(name="frequency")
)
```

What does this code do?
Counts each dependent–predicate XPOS pattern in each text.

Example

$$\texttt{NNG+JKS} \xrightarrow{\texttt{nsubj}} \texttt{VV+EC + VX+EC}$$

3.4.8. Compare morphosyntactic patterns

```
morphosyntactic_comparison = (
    morphosyntactic_patterns
    .pivot_table(
        index="morphosyntactic_pattern",
        columns="document",
        values="frequency",
        fill_value=0,
        aggfunc="sum",
    )
    .reset_index()
)

morphosyntactic_comparison["total"] = (
    morphosyntactic_comparison["text_5"]
    + morphosyntactic_comparison["text_6"]
)

display(
    morphosyntactic_comparison
    .drop(columns="total")
)
```

What does this code do?
Places the pattern frequencies from Texts 5 and 6 side by side.

3.4.9. Visualize the patterns

```
TOP_N = 10

plot_data = (
    morphosyntactic_comparison
    .head(TOP_N)
    .sort_values(
        "total",
        ascending=True)
    .set_index(
        "morphosyntactic_pattern"))

ax = plot_data[
    ["text_5", "text_6"]
].plot(
    kind="barh",
    figsize=(10, 6),
)

plt.tight_layout()
plt.show()
```

What does this code do?

Visualizes the ten most frequent morphosyntactic patterns across the two texts.

You may change

Modify `TOP_N` to show more or fewer patterns.

Activity 4. Interpret the concordance lines

1. Which dependency relation did you select?
2. Which dependent lemma did you inspect?
3. How many concordance lines were found?
4. Which predicate contexts occur with the selected dependent?
5. Which morphosyntactic pattern is most frequent?
6. Are the frequent patterns similar or different across Texts 5 and 6?
7. What information would be missed if you examined only `head_lemma`?
8. Inspect one recurring pattern in the original sentences:
   - If yes, how would you interpret it?
   - If no, what additional information would you add to the concordance line? How would you like to edit/rewrite the codes to do that?
9. How could dependency-based concordancing support research questions in your own area of interest?

Conclusion

Evaluation on L2 Korean

- Current NLP tools achieve overall high accuracy in morphosyntactic tagging and dependency parsing of L2 Korean.

- Note that their performance varies across different tags.

Metric	Stanza	Trankit
LEMMA	95.64	88.84
XPOS	89.72	91.81
UAS	85.53	92.28
LAS	80.36	89.13

Table 2: Performance comparison (F1 scores) of fine-tuned Stanza and Trankit models on the L2K-UD test dataset

Sung & Shin (2026). Parser Agreement and Disagreement in L2 Korean Universal Dependencies

Application: Korean Lexical Complexity Analyzer

- The Korean Lexical Complexity Analyzer automatically measures multiple dimensions of lexical complexity in Korean texts.
- It uses morphosyntactic annotation to identify lexical items, which supports the calculation of measures such as lexical diversity, sophistication, and compositionality.

Demo: `huggingface.co/spaces/hksung/KLC-demo`

GitHub: `github.com/hksung/Korean-lexical-complexity-analyzer`

Thank you!

Contact: sungh@hawaii.edu

Appendix

Sejong tag set

- The Sejong tag set provides Korean-specific part-of-speech labels at the morpheme level
- It distinguishes fine-grained categories such as common nouns, case particles, verb endings, and auxiliary predicates
- See the full tag set and descriptions:
  `https://nlpxl2korean.github.io/UD-KSL/xpos/`

UD tags

- See the full list of dependency relations and examples (*tailored to Korean*): `https://nlpxl2korean.github.io/UD-KSL/ud/`

L2K-ARG corpus

- 484 argumentative essays written by L2 Korean learners
- Provides C-test scores and selected human-rated writing scores
- Can be accessed here: `https://github.com/NLPxL2Korean/L2K-ARG`